

✓ Data Wrangling 1

import the requiried libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Load the dataset

```
df = pd.read_csv('weatherAUS.csv')
```

```
#data.head displays top 5 records by default
df.head(10)
```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Su
0	5/18/2009	Hobart	5.1	14.3	0.0	1.8	
1	07-03-2009	Launceston	1.1	14.5	0.4	NaN	
2	2/18/2010	Williamtown	19.7	26.2	0.0	7.2	
3	03-04-2010	PerthAirport	16.6	28.0	0.0	9.0	
4	09-09-2010	GoldCoast	14.6	25.3	0.0	NaN	
5	9/28/2011	Mildura	15.7	27.3	0.0	11.2	
6	10/13/2011	Nuriootpa	9.8	23.9	0.0	4.4	
7	04-08-2010	NorahHead	18.7	27.6	3.0	NaN	
8	10/19/2009	Townsville	23.6	30.1	0.0	11.4	
9	2/23/2010	MountGambier	8.5	22.9	1.4	4.2	

10 rows x 24 columns

```
#It display bottom 5 records
df.tail()
```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	S
36876	6/14/2011	Perth	10.1	14.2	7.4	3.4	
36877	9/24/2010	PerthAirport	6.3	26.0	0.0	5.0	
36878	05-06-2011	Darwin	20.5	31.7	0.0	7.0	
36879	10-12-2010	Perth	6.3	23.3	0.0	5.2	
36880	12/22/2010	Nuriootpa	9.4	26.3	0.0	4.8	

5 rows x 24 columns

Data Processing

```
#display information about dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36881 entries, 0 to 36880
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  36881 non-null  object
1   Location              36881 non-null  object
2   MinTemp               36543 non-null  float64
3   MaxTemp              36639 non-null  float64
4   Rainfall              36255 non-null  float64
5   Evaporation           24035 non-null  float64
6   Sunshine              23317 non-null  float64
7   WindGustDir           33513 non-null  object
8   WindGustSpeed         33520 non-null  float64
9   WindDir9am            34072 non-null  object
10  WindDir3pm            35919 non-null  object
11  WindSpeed9am          36219 non-null  float64
12  WindSpeed3pm          36235 non-null  float64
13  Humidity9am           36311 non-null  float64
14  Humidity3pm           36370 non-null  float64
15  Pressure9am           33309 non-null  float64
16  Pressure3pm           33329 non-null  float64
17  Cloud9am              24381 non-null  float64
18  Cloud3pm              23899 non-null  float64
19  Temp9am               36394 non-null  float64
20  Temp3pm               36437 non-null  float64
21  RainToday             36255 non-null  object
22  RISK_MM               36261 non-null  float64
23  RainTomorrow          36261 non-null  object
dtypes: float64(17), object(7)
memory usage: 6.8+ MB
```

```
# give the column name  
df.columns
```

```
Index(['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall',  
      'Evaporation',  
      'Sunshine', 'WindGustDir', 'WindGustSpeed', 'WindDir9am',  
      'WindDir3pm',  
      'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am',  
      'Humidity3pm',  
      'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm',  
      'Temp9am',  
      'Temp3pm', 'RainToday', 'RISK_MM', 'RainTomorrow'],  
      dtype='object')
```

Descriptive Statistics: It is used to summarize and describe the features of data in a meaningful way to extract insights.

```
# It gives descriptive statistics of numerical variable
print(df.describe(exclude=[object]))
```

	MinTemp	MaxTemp	Rainfall	Evaporation	S
count	36543.000000	36639.000000	36255.000000	24035.000000	23317
mean	12.235966	22.939491	2.554823	5.189544	7
std	6.389832	6.988266	9.081329	3.765427	3
min	-8.500000	-3.100000	0.000000	0.000000	0
25%	7.600000	17.800000	0.000000	2.600000	4
50%	12.100000	22.300000	0.000000	4.600000	8
75%	17.000000	27.900000	0.800000	7.000000	10
max	30.500000	48.100000	371.000000	81.200000	14
	WindGustSpeed	WindSpeed9am	WindSpeed3pm	Humidity9am	Hum
count	33520.000000	36219.000000	36235.000000	36311.000000	3637
mean	40.390573	14.217427	18.874928	69.146953	5
std	13.550613	8.997003	8.791849	18.300754	2
min	7.000000	0.000000	0.000000	2.000000	
25%	31.000000	7.000000	13.000000	58.000000	3
50%	39.000000	13.000000	19.000000	70.000000	5
75%	48.000000	20.000000	24.000000	83.000000	6
max	135.000000	87.000000	87.000000	100.000000	10
	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	
count	33309.000000	33329.000000	24381.000000	23899.000000	36394
mean	1017.283152	1014.967769	4.331488	4.462321	16
std	7.177877	7.077594	2.848750	2.663067	6
min	982.900000	980.200000	0.000000	0.000000	-5
25%	1012.500000	1010.100000	1.000000	2.000000	12
50%	1017.300000	1015.000000	5.000000	5.000000	16
75%	1022.200000	1019.800000	7.000000	7.000000	21
max	1041.000000	1039.600000	9.000000	8.000000	37
	Temp3pm	RISK_MM			
count	36437.000000	36261.000000			
mean	21.485808	2.554748			
std	6.820848	8.940267			
min	-5.100000	0.000000			
25%	16.500000	0.000000			
50%	20.900000	0.000000			
75%	26.200000	0.800000			
max	46.100000	367.600000			

```
print(df.describe(include=[object]))
```

	Date	Location	WindGustDir	WindDir9am	WindDir3pm	RainTomorrow
count	36881	36881	33513	34072	35919	36261
unique	1473	46	16	16	16	2
top	03-03-2009	Canberra	SE	N	SE	No
freq	42	1085	2403	2862	2939	27732

Finding Categorical and Numerical Features in a Dataset:

Categorical features in Dataset:

```
categorical_features= [column_name for column_name in df.columns if
print("Number of Categorical Features: {}".format(len(categorical_f
print("Categorical Features: ",categorical_features)
```

```
Number of Categorical Features: 7
Categorical Features:  ['Date', 'Location', 'WindGustDir', 'WindDir9
```

Numerical features in Dataset:

```
numerical_features = [column_name for column_name in df.columns if
print("Number of Numerical Features: {}".format(len(numerical_featu
print("Numerical Features: ",numerical_features)
```

```
Number of Numerical Features: 17
Numerical Features:  ['MinTemp', 'MaxTemp', 'Rainfall', 'Evaporatio
```

```
df.isnull().sum()
```

	0
Date	0
Location	0
MinTemp	338
MaxTemp	242
Rainfall	626
Evaporation	12846
Sunshine	13564
WindGustDir	3368
WindGustSpeed	3361
WindDir9am	2809
WindDir3pm	962
WindSpeed9am	662
WindSpeed3pm	646
Humidity9am	570
Humidity3pm	511
Pressure9am	3572
Pressure3pm	3552
Cloud9am	12500
Cloud3pm	12982
Temp9am	487
Temp3pm	444
RainToday	626
RISK_MM	620
RainTomorrow	620

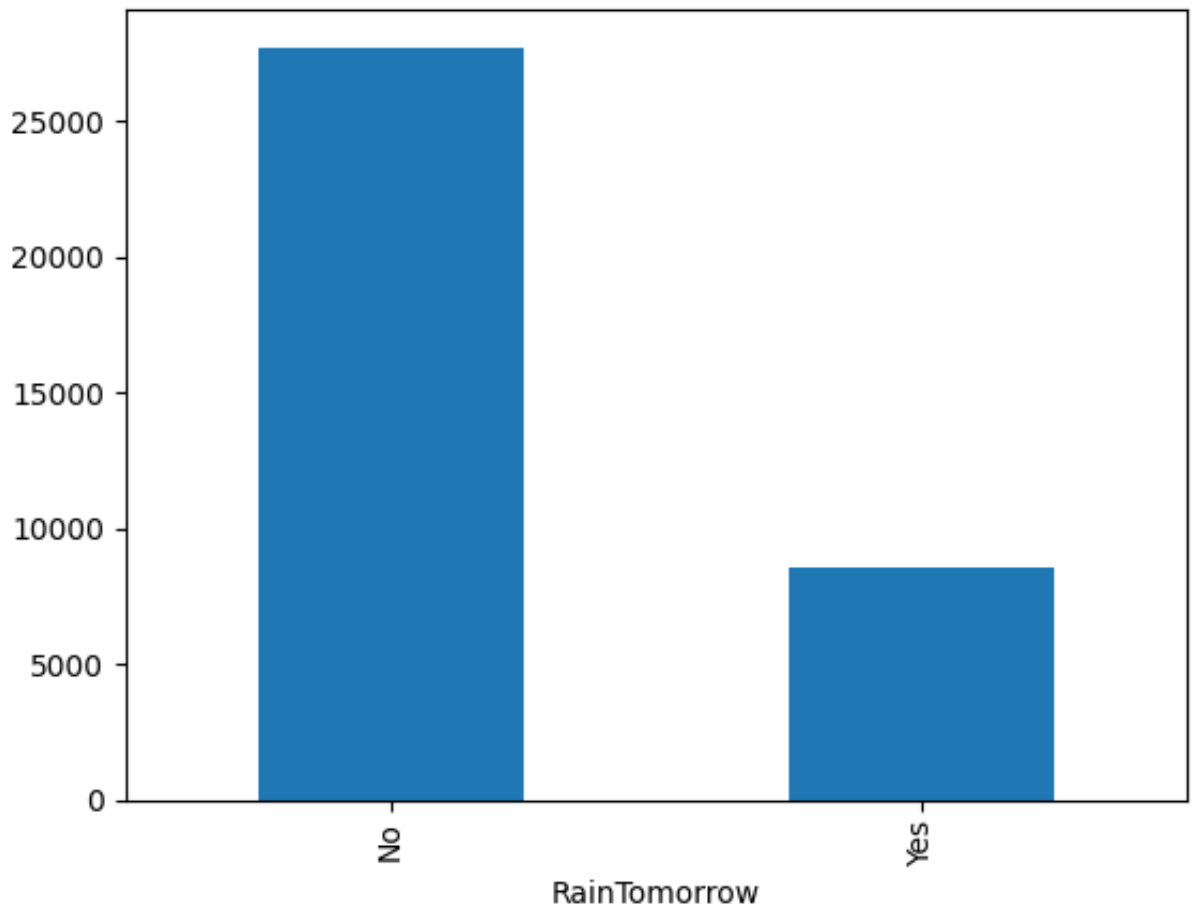
dtype: int64

```
df.shape
```

```
(36881, 24)
```

```
df['RainTomorrow'].value_counts().plot.bar()
```

<Axes: xlabel='RainTomorrow'>



```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

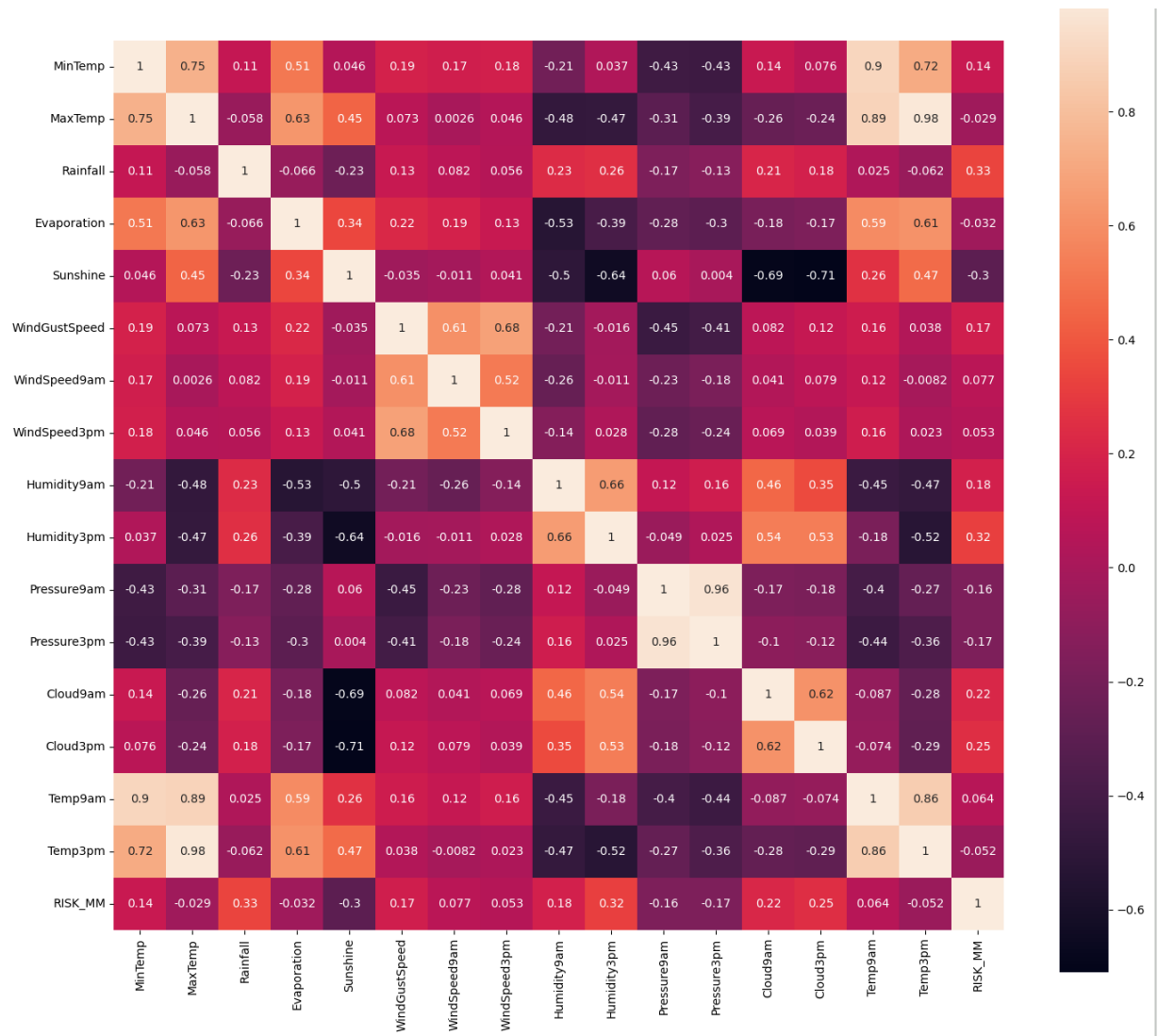
# Select only numeric columns
df = df.select_dtypes(include=[np.number])

plt.figure(figsize=(17,15))

corre = sns.heatmap(df.corr(), square=True, annot=True)

corre.set_xticklabels(corre.get_xticklabels(), rotation=90)

plt.show()
```

✓ Handling Missing values in Categorical Features:

```
categorical_features = [column_name for column_name in df.columns if  
df[column_name].isnull().sum() > 0]
```

0

dtype: float64

```
# Imputing missing values in categorical features using mode
```

```
categorical_features_with_null = [  
    feature for feature in categorical_features  
    if df[feature].isnull().sum() > 0  
]
```

```
for each_feature in categorical_features_with_null:  
    mode_val = df[each_feature].mode()[0]  
    df[each_feature].fillna(mode_val, inplace=True)
```

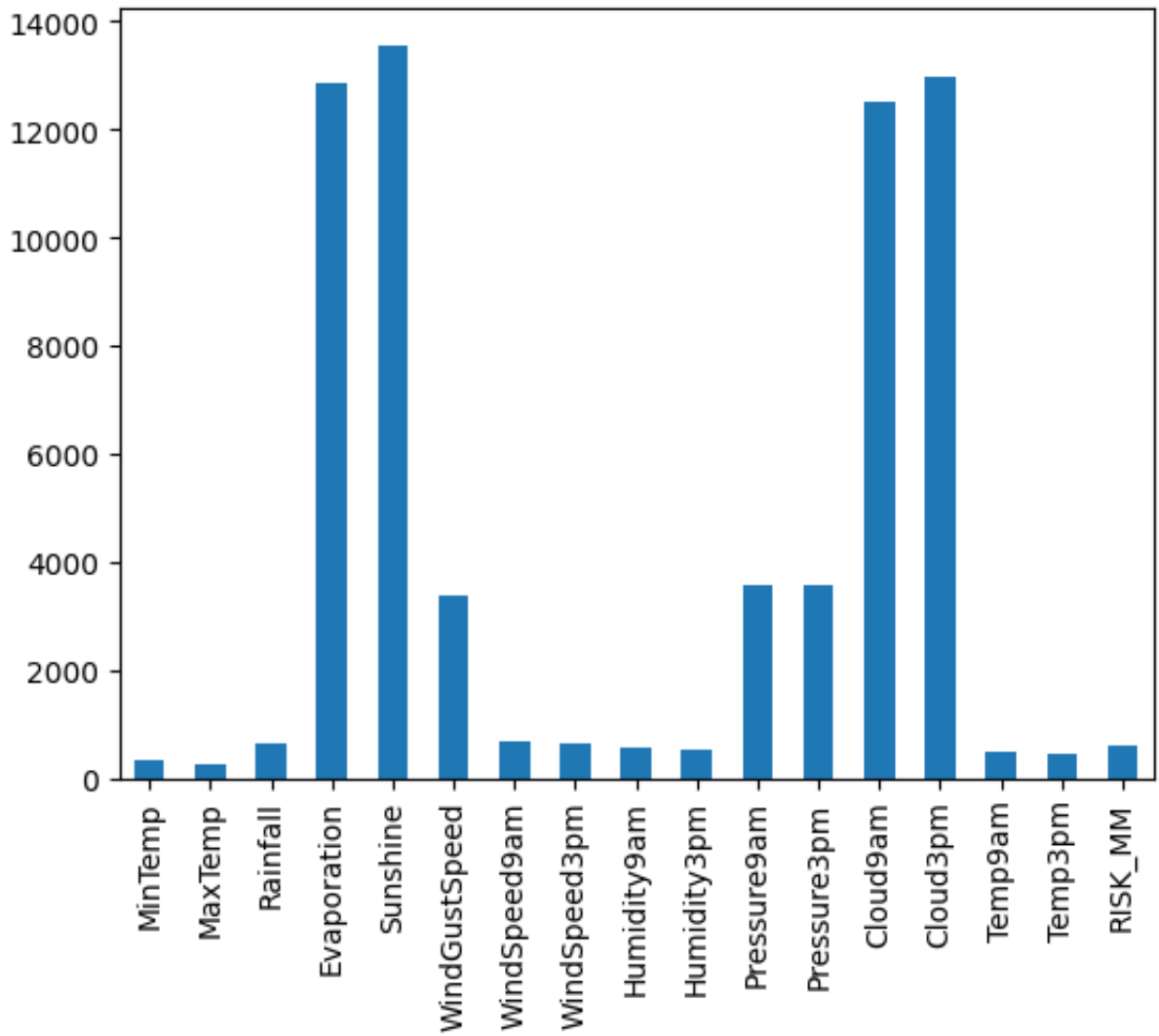
```
# Handling Missing values in Numerical features:  
numerical_features = [column_name for column_name in df.columns if  
df[numerical_features].isnull().sum()
```

	0
MinTemp	338
MaxTemp	242
Rainfall	626
Evaporation	12846
Sunshine	13564
WindGustSpeed	3361
WindSpeed9am	662
WindSpeed3pm	646
Humidity9am	570
Humidity3pm	511
Pressure9am	3572
Pressure3pm	3552
Cloud9am	12500
Cloud3pm	12982
Temp9am	487
Temp3pm	444
RISK_MM	620

dtype: int64

```
df[numerical_features].isnull().sum().plot.bar()
```

<Axes: >



```
# Outlier Treatment to remove outliers from Numerical Features

features_with_outliers = [
    'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
    'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm'
]

for feature in features_with_outliers:
    q1 = df[feature].quantile(0.25)
    q3 = df[feature].quantile(0.75)
    IQR = q3 - q1

    lower_limit = q1 - (IQR * 1.5)
    upper_limit = q3 + (IQR * 1.5)

    df.loc[df[feature] < lower_limit, feature] = lower_limit
    df.loc[df[feature] > upper_limit, feature] = upper_limit
```

Now, numerical features are free from outliers.

✓ Let's Impute missing values in numerical features using mean

```
numerical_features_with_null= [feature for feature in numerical_fea
for feature in numerical_features_with_null:
    mean_value =df[feature].mean()
    df[feature].fillna(mean_value,inplace=True)
```

/tmp/ipykernel_7073/2208545658.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series resulting in 0 rows being updated. The behavior will change in pandas 3.0. This inplace method will never work.

For example, when doing 'df[col].method(value, inplace=True)', try using df[col] = df[col].method(value)

```
df[feature].fillna(mean_value,inplace=True)
```

```
df.isnull().sum()
```

	0
MinTemp	0
MaxTemp	0
Rainfall	0
Evaporation	0
Sunshine	0
WindGustSpeed	0
WindSpeed9am	0
WindSpeed3pm	0
Humidity9am	0
Humidity3pm	0
Pressure9am	0
Pressure3pm	0
Cloud9am	0
Cloud3pm	0
Temp9am	0
Temp3pm	0
RISK_MM	0

dtype: int64

✓ Encoding

```
def encode_data(feature_name):
    """
    This function takes feature name and returns mapping dictionary
    """

    mapping_dict = {}

    unique_values = list(df[feature_name].unique())

    for idx in range(len(unique_values)):
        mapping_dict[unique_values[idx]] = idx

    return mapping_dict
```

```
df['RainToday'] = df['RainToday'].replace({'No':0, 'Yes':1})
df['RainTomorrow'] = df['RainTomorrow'].replace({'No':0, 'Yes':1})

df['WindGustDir'] = df['WindGustDir'].replace(encode_data('WindGustDir'))
df['WindDir9am'] = df['WindDir9am'].replace(encode_data('WindDir9am'))
df['WindDir3pm'] = df['WindDir3pm'].replace(encode_data('WindDir3pm'))
df['Location'] = df['Location'].replace(encode_data('Location'))
```

```
/tmp/ipykernel_7073/2237296444.py:1: FutureWarning: Downcasting behavior
df['RainToday'] = df['RainToday'].replace({'No':0, 'Yes':1})
/tmp/ipykernel_7073/2237296444.py:2: FutureWarning: Downcasting behavior
df['RainTomorrow'] = df['RainTomorrow'].replace({'No':0, 'Yes':1})
/tmp/ipykernel_7073/2237296444.py:4: FutureWarning: Downcasting behavior
df['WindGustDir'] = df['WindGustDir'].replace(encode_data('WindGustDir'))
/tmp/ipykernel_7073/2237296444.py:5: FutureWarning: Downcasting behavior
df['WindDir9am'] = df['WindDir9am'].replace(encode_data('WindDir9am'))
/tmp/ipykernel_7073/2237296444.py:6: FutureWarning: Downcasting behavior
df['WindDir3pm'] = df['WindDir3pm'].replace(encode_data('WindDir3pm'))
/tmp/ipykernel_7073/2237296444.py:7: FutureWarning: Downcasting behavior
df['Location'] = df['Location'].replace(encode_data('Location'))
```

```

encode_data('RainToday')
df['RainTomorrow']
df['WindGustDir']
df['WindDir9am']
df['WindDir3pm']
df['Location']

```

	Location
0	0
1	1
2	2
3	3
4	4
...	...
36876	35
36877	3
36878	12
36879	35
36880	6

36881 rows × 1 columns

dtype: int64

```

df['RainToday'] = df['RainToday'].replace({'No':0, 'Yes':1})
df['RainTomorrow'] = df['RainTomorrow'].replace({'No':0, 'Yes':1})
df['WindGustDir'] = df['WindGustDir'].replace(encode_data('WindGust
df['WindDir9am'] = df['WindDir9am'].replace(encode_data('WindDir9am
df['WindDir3pm'] = df['WindDir3pm'].replace(encode_data('WindDir3pm
df['Location'] = df['Location'].replace(encode_data('Location'))

```

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

```



```
# Select only numeric columns
df_numeric = df.select_dtypes(include=[np.number])

plt.figure(figsize=(20,20))
sns.heatmap(df_numeric.corr(), linewidths=0.5, annot=False, fmt=".2")
plt.show()
```

